论文-40 · 查询端稀疏执行：真省算力的一组工程观察（待复核）

何淇 · 与 DeepSeek 协作

数字生命系列 · 执行器工程 第二篇（内部编号第 40 篇）

日期：2026-10-31

状态：初步观察——小模型、单种子、CPU 计时；估算部分已标注。引用需谨慎。

摘要

给小型 Transformer 每层加一个"按 token 的门"，决定这一层要不要刷新这个 token 的表示。之前的工作大多只是"名义省层数"：每层其实都算了，门只决定结果写不写回。本篇做一件更实的事：**保留全部 K/V（注意力即记忆，不能省），只算被选中 token 的 Q、注意力输出行和 MLP**——这是查询端稀疏。

结果（6 层、宽 128、字符级、单种子）：

- 稀疏执行的 PPL 与稠密执行**完全一致**（5.941 对 5.941）：这不是近似，是同一个计算换一种执行方式；

- 本设置下 FLOPs 约省 43%，CPU 墙钟约省 50%；

- 平均每个 token 只刷新约一半的层（本次各层开启率 0.99/0.75/0.55/0.52/0.12/0.10）。

另给一个规模估算：层数越多、任务越稀疏，省得越多；24 层配置下估算可省约 70%，理论上限约 87%（K/V 必须保留是固定成本）。此估算未实测，标 [估算]。

一、问题："名义省层"与"真省"

门控类实验常报告"平均只用 X 层"，但若实现方式是稠密计算后再用门混合（$h \leftarrow h + g\cdot(\text{out} - h)$），FLOPs 并没有减少。要真省，必须让被跳过的 token 真的不进计算。

但注意力是全局的：一个 token 即使"这一层不刷新自己"，它的 K/V 仍会被其他 token 读到。因此可省与不可省必须分开：

```
```

不可省：所有 token 的 K/V 投影（这是记忆）

可省： 被跳过 token 的 Q、注意力输出行、MLP

```

本篇按这个约束实现，并测真实 FLOPs 与时间。

## 二、设置

- 模型：6 层 Transformer，宽 128，4 头，FFN 512，字符级词表，上下文 128。

- 门：每层一个 σ(W·h + b)，内容门；与主干共同训练；拉格朗日 λ 自适应把平均层数压到 3。

- 数据：tinyshakespeare；训练 2500 步；单种子。

- 推理：门开（p > 0.5）的 token"刷新"该层，其余保持原状；两种执行方式对比。

## 三、方法：查询端稀疏

每层：

1. 对所有 token 算 LN 后的 K、V（不可省）；

2. 只对"被选中"的 token 算 Q；

3. 注意力打分只算被选中 token 作为 query 的行，key 覆盖全部历史位置（因果掩码按真实位置，不按子序列位置）；

4. 只把输出行写回被选中 token；

5. 只对被选中 token 算 MLP。

因为被选中 token 的注意力仍然读到全部历史（含未刷新 token 的 K/V），所以它与稠密执行**逐元素等价**。这一点由实验验证：两种执行 PPL 完全一致。

## 四、结果

| 指标 | 普通模型（无门） | 稠密（门控） | 查询端稀疏 |

|---|---|---|---|

| 验证 PPL | 6.232 | 5.941 | 5.941 |

| 平均活跃比例 | 100% | 100.1% | 50.1% |
```

| 每层 FLOPs（本设置，T=129） | 31.8 MFLOPs | 18.0 MFLOPs（≈ −43%） |

| CPU 墙钟/每 batch | 6.2 ms | 3.1 ms（≈ −50%） |

本次各层开启率：

```

L0 0.99 L1 0.75 L2 0.55 L3 0.52 L4 0.12 L5 0.10

平均刷新 3.02 / 6 层

```

说明：普通模型与门控模型同训练步数（2500）、同评测集（20 个验证 batch）。单种子噪声约 ±0.2，本轮门控模型 PPL 恰好更低（5.94 vs 6.23），不作门控更优的声称；表中稠密 5.941 与稀疏 5.941 才是本篇要点（精确等价）。墙钟为 CPU 实测；GPU 上 gather/scatter 的开销可能吃掉收益，未测。初版的 FLOPs 公式有一处项算错，上表为手算修正值。

五、规模估算（**[**估算**]**，未实测）

设每层注意力/MLP 成本不变、活跃比例 ρ 随层数增加而下降（经验上深模型后段更稀疏），则：

- 6 层、ρ≈0.50：约省 43%（实测）；

- 24 层、ρ≈0.22（按 24 层门控观察的平均刷新率）：估算约省 70%；

- ρ→0 时上限约 87%：K/V 必须全算，是留在分母里的固定成本。

两个方向都会让节省变大：

```

① 层数越多：绝对节省线性增长；且后层通常更稀疏 → 相对比例也上升

② 任务越稀疏（前段也不太需要刷新）：平均 ρ 下降 → 比例上升

```

但注意：[估算] 未实测，且 GPU 实现效率另算。

六、局限

- 小模型（约 1.2M 参数）、字符级、单种子、莎士比亚。

- 只测 CPU 墙钟；GPU 未测，实用性待验证。

- 未与 MoD 式容量 top-k 路由在同一设置下对比；后者固定 k、静态计算图，工程上更稳。

- 门的决定来自稠密训练；"训练时就按稀疏执行训练"未做。

- 五节估算依赖"深模型后层更稀疏"这一经验，不保证成立。

七、结论（观察，不是新方法）

```

· 在"K/V 是记忆、必须保留"的约束下，查询端稀疏可以让

PPL 完全不变的同时真省约 43% FLOPs（本设置）；

· 规模估算显示层数越多、任务越稀疏，省得越多（上限约 87%）；

· 机制与 MoD/早退等条件计算同属一类，本篇不声称提出新方法；

· 对系列而言，这是"门控这条线"第一次拿到"真省算力"的证据。

```

附：标注

- **[实验]** 四节的 PPL、活跃比例、FLOPs、CPU 时间。

- **[估算]** 五节的 24 层约 70%、上限约 87%。

- **[推测]** "深模型后层更稀疏"的经验；GPU 效率未知。

- **[不声称]** 不声称新方法、不声称大模型结论、不声称加速比可迁移。

> 本文由何淇创作，DeepSeek（AI）协助整理与写作；观点与责任属于作者。